Regression Analysis using R – Practicals 1 to 6 (Questions and Codes)

T.Y.B.Sc. I.T. (NEP 2020) | Minor-II: (1085213) Practical in Regression Analysis using R Programming

Practical No. 01

The Practical 1 PDF was not among the uploaded files, so its questions and codes are not included here. Share it and it can be added.

Practical No. 02 – Graphs using R

Q.1. Line graph

The number of students attending an online lecture over five days is given below:

Day	Students
Monday	40
Tuesday	45
Wednesday	50
Thursday	48
Friday	60

Create a line graph to represent the data.

```
cat('\014')
days<-c("Monday","Tuesday","Wednesday","Thursday","Friday")
students<-c(40, 45, 50, 48, 60)
plot(students,
     type="o",
     xaxt="n",
     main="Students Attending Online Lecture",
     xlab="Day",
     ylab="Number of Students",
     pch=16)
axis(1,at=1:5,labels=days)
```

Q.2. Bar graph

The number of students enrolled in different IT specializations is:

- Data Science – 40
- Cyber Security – 35
- AI – 45
- Cloud Computing – 30

Create a bar graph.

```
cat('\014')
specialization<-c("Data Science","Cyber Security",
                  "AI","Cloud Computing")
students<-c(40, 35, 45, 30)
bar_pos<-barplot(students,
        names.arg=specialization,
        main="Students in IT Specializations",
        xlab="Specialization",
        ylab="Number of Students",
        ylim=c(0,50),
        col="cyan")

text(x=bar_pos,y=students,labels=students,pos=3)
```

Q.3. Pie chart

A student spends weekly study time on different IT subjects as follows:

Subject	Hours
Programming	10
Database	6
Networking	5
Mathematics	4
Other	3

Create a pie chart.

```
cat('\014')
subjects<-c("Programming","Database","Networking","Mathematics","Other")
hours<-c(10,6,5,4,3)

#Pie Chart with values
lab<-paste(subjects,"\n",hours)
pie(hours,
    labels=lab,
    main="Weekly Study Time",
    col=rainbow(length(hours)))

#Pie Chart with Percentage
pc<-round(hours/sum(hours)*100,2)
lab<-paste(subjects,"\n",paste0("(", pc, "%)"))
pie(hours,
    labels=lab,
    main="Weekly Study Time",
    col=rainbow(length(hours)))
```

Q.4. Histogram

The following marks were obtained by 20 students in a programming test:

45, 55, 60, 62, 65, 67, 70, 72, 74, 75, 76, 78, 80, 82, 84, 85, 88, 90, 92, 95

Create a histogram to study the distribution of marks.

```
cat('\014')
marks<-c(45, 55, 60, 62, 65, 67, 70, 72, 74, 75,
         76, 78, 80, 82, 84, 85, 88, 90, 92, 95)

h<-hist(marks,
     main="Distribution of Programming Marks",
     xlab="Marks",
     ylab="Frequency",
     ylim=c(0,7),
     breaks=5,
     col="lightgreen")

text(x=h$mids,y=h$counts,labels=h$counts,pos=3)
```

Q.5. Scatter plot

The following data represents the relationship between study hours and programming marks.

Study Hours	Marks
2	45
3	50
4	55
5	65
6	70
7	75
8	82
9	88

Create a scatter plot.

```
cat('\014')
study_hours<-c(2,3,4,5,6,7,8,9)
marks<-c(45,50,55,65,70,75,82,88)

plot(study_hours, marks,
     main="Study Hours vs Programming Marks",
     xlab="Study Hours",
     ylab="Marks",
     xlim=c(0,10),
     ylim=c(0,100),
     pch=19,
     col="blue")
```

Q.6. Box plot

The marks obtained by students in three subjects are given below. Create a box plot to compare their distributions.

Subject									
Programming	55	60	65	68	70	72	75	80	85
Database	50	55	60	62	65	68	70	75	78
Networking	45	50	55	58	60	63	67	70	72

```
cat('\014')
programming<-c(55, 60, 65, 68, 70, 72, 75, 80, 85)
database<-c(50, 55, 60, 62, 65, 68, 70, 75, 78)
networking<-c(45, 50, 55, 58, 60, 63, 67, 70, 72)

marks<-data.frame(programming,database,networking)

boxplot(marks,
        main="Subject-wise Marks Distribution",
        xlab="Subjects",
        ylab="Marks",
        col=c("lightblue","lightgreen","lightyellow"))
```

Q.7. Multiple line graph

The number of software projects completed by two teams over four months is given below.

Month	Team A	Team B
January	5	4
February	7	6
March	8	9
April	10	11

Create a multiple line graph to compare the performance of the two teams.

```
cat('\014')
months <- c("January", "February", "March", "April")
teamA <- c(5, 7, 8, 10)
teamB <- c(4, 6, 9, 11)

#Plot Team A Line
plot(teamA,
     type = "o",
     pch = 16,
     col = "red",
     xaxt = "n",
     ylim = c(0, 12),
     main = "Projects Completed by Teams",
     xlab = "Month",
     ylab = "Number of Projects"
     )
axis(1, at = 1:4, labels = months)

#Plot Team B Line
lines(teamB,
      type = "o",
      pch = 8,
      col = "blue")

#Legends
legend("topleft",
```

```
       legend = c("Team A", "Team B"),
       col = c("red","blue"),
       pch = c(16, 8),
       lty = 1)
```

Q.8. Horizontal bar graph

The number of users of different programming languages in a class is:

Language	Users
Python	45
R	30
Java	35
C++	25
JavaScript	40

Create a horizontal bar graph.

```
cat('\014')
languages <- c("Python", "R", "Java", "C++", "JavaScript")
users <- c(45, 30, 35, 25, 40)
par(mar = c(5, 7 ,5, 2)) # To adjust margin for Y-axis tick labels
par(mgp = c(5, 1, 0)) # To adjust position of Y-axis title
bar_pos<-barplot(users,
                 names.arg = languages,
                 horiz = TRUE,
                 main = "Programming Languages Used by Students",
                 xlab = "Number of Students",
                 ylab = "Programming Language",
                 col = "orange",
                 xlim = c(0,50),
                 las = 1)
text(x = users + 2,
     y = bar_pos,
     labels = users,
     pos = 4)
```

Q.9. Dot chart

The following data represents the number of hours spent by students on different IT activities.

Activity	Hours
Coding	12
Reading	7
Project	10
Testing	6
Documentation	5

Create a dot chart.

```
cat('\014')
activity <- c("Coding",
              "Reading",
              "Project",
              "Testing",
              "Documentation")

hours <- c(12, 7, 10, 6, 5)

dotchart(hours,
         labels = activity,
         main = "Time Spent on IT Activities",
         xlab = "Hours")
```

Q.10. Line graph with points and grid lines

The following data represents the monthly internet data usage of a student.

Month	Data Usage (GB)
January	80
February	95
March	110

Month	Data Usage (GB)
April	105
May	125
June	140

Create a line graph with points and grid lines.

```
cat('\014')
month <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun")
usage <- c(80, 95, 110, 105, 125, 140)

plot(usage,
     type = "o",
     xaxt = "n",
     main = "Monthly Internet Data Usage",
     xlab = "Month",
     ylab = "Data Usage (GB)",
     pch = 19)

axis(1, at = 1:6, labels = month)

grid()
```

Practical No. 03 – Diagrams using R

Q.1. Create the following diagram in R (Computer → Hardware/Software tree)

Diagram: Computer points to Hardware and Software; Hardware points to CPU and Memory; Software points to OS and Application.

```
cat('\014')
library(DiagrammeR)
grViz("
digraph {
  Computer->Hardware
  Computer->Software
  Hardware->CPU
  Hardware->Memory
  Software->OS
  Software->Application
}
")
```

Q.2. Create the following diagram in R (Marks pass/fail flow)

Diagram: Start → Marks → Check; Check → Pass (label: Marks >= 40) and Check → Fail (label: Marks < 40); Pass → End and Fail → End.

```
library(DiagrammeR)
grViz("
digraph {
  Start->Marks
  Marks->Check
  Check->Pass [label='Marks >= 40']
  Check->Fail [label='Marks < 40']
  Pass->End
  Fail->End
}
")
```

Q.3. Create the following diagram in R (Directed Graph with igraph)

Diagram: a directed graph with vertices A, B, C, D, E (cyan vertices with blue frames), titled "Directed Graph".

```
cat('\014')
library(igraph)
g<-graph_from_literal(
  A+-B,
  A-+C,
  B-+D,
  C+-D,
```

```
  D-+E
)
set.seed(10)
plot(g,
     layout=layout_with_fr(g),
     vertex.color="cyan",
     vertex.frame.color="blue",
     vertex.frame.width=3,
     vertex.size=30,
     vertex.label.color="black",
     edge.color="black",
     edge.width=2,
     main = "Directed Graph")
```

Q.4. Create the following diagram in R (Login flowchart)

Diagram: START → Enter Username and Password → Validate Login (diamond); Valid → Dashboard, Invalid → Invalid Login; both → END.

```
library(DiagrammeR)

grViz("
digraph {

  Start [label='START', shape=oval]
  Login [label='Enter Username\\nand Password', shape=box]
  Validate [label='Validate Login', shape=diamond]
  Dashboard [label='Dashboard', shape=box]
  Error [label='Invalid Login', shape=box]
  End [label='END', shape=oval]

  Start -> Login
  Login -> Validate
  Validate -> Dashboard [label='Valid']
  Validate -> Error [label='Invalid']
  Dashboard -> End
  Error -> End
}
")
```

Q.5. Create the following diagram in R (Web architecture with dashed style)

Diagram: Web Browser → Web Server → Application Server → Database Server and Payment Gateway, all dashed rectangles with dashed edges.

```
cat('\014')
library(DiagrammeR)
grViz("
digraph{
      node[shape=rectangle,style=dashed]
      edge[style=dashed]
      'Web Browser'->'Web Server'
      'Web Server'->'Application\nServer'
      'Application\nServer'->'Database\nServer'
      'Application\nServer'->'Payment\nGateway'
}"
)
```

Practical No. 04 – Simple Linear Regression

Q.1. Study hours vs marks

A college teacher wants to study whether the number of hours a student spends studying has an effect on the marks obtained in a programming examination. The teacher records the following data for 10 students.

Study Hours	2	3	4	5	6	7	8	9	10	11
Marks	45	50	54	61	65	70	74	80	85	88

Perform the following tasks in R:

1. Create a data frame containing the above data.
2. Fit a simple linear regression model with Marks as the dependent variable and Study Hours as the independent variable.
3. Display the summary of the regression model.
4. Obtain the regression equation.
5. Predict the marks of a student who studies for 7.5 hours.
6. Plot the data points and regression line.

```
cat('\014')
study_hrs<-c(2,3,4,5,6,7,8,9,10,11)
marks<-c(45,50,54,61,65,70,74,80,85,88)
df<-data.frame(study_hrs,marks)
model<-lm(marks~study_hrs)
print(summary(model))
eq<-coef(model)
b0<-eq[1]
b1<-eq[2]
cat("Regression Equation : y=",b0,"+",b1,"x")
ans<-predict(model,data.frame(study_hrs=7.5))
cat("\nPredicted Marks of a student who studies for 7.5 hours = ",ans)
plot(study_hrs,marks,
     main="Marks vs Study Hours",
     xlab="Study Hours",
     ylab="Marks",
     xlim=c(0,12),
     ylim=c(0,100))
abline(model,col="red")
```

Q.2. Women dataset

The women dataset contains the height and weight of women. A health researcher wants to investigate whether height can be used to predict weight.

Perform the following tasks in R:

1. Load the women dataset.
2. Display the dataset.
3. Fit a simple linear regression model.
4. Identify the independent and dependent variables.
5. Obtain the regression equation.
6. Predict the weight of a woman with a height of 65 inches.
7. Display actual values, predicted values and residuals.

```
cat('\014')
data(women)
print(women)
model<-lm(weight~height,data=women)
print(summary(model))
eq<-coef(model)
```

```
b0<-eq[1]
b1<-eq[2]
cat("Regression Equation : y=",b0,"+",b1,"x")
ans<-predict(model,data.frame(height=65))
cat("\nPredicted weight of a woman with a height of 65 inches = ",ans,"\n\n")
predicted<-fitted(model)
res<-residuals(model)
result<-data.frame(
  Actual_Height=women$height,
  Actual_Weight=women$weight,
  Predicted_Weight=predicted,
  Residuals=res
)
print(result)
```

Q.3. RAM vs performance score

An IT department wants to determine whether increasing the RAM of a computer improves its application performance score.

RAM (GB)	4	6	8	10	12	14	16	18
Performance Score	45	52	60	67	73	78	84	88

Perform the following tasks in R:

1. Enter the data into R.
2. Fit a simple linear regression model.
3. Obtain the intercept and slope.
4. Predict the website loading time when 75 users access the website simultaneously. (As printed in the PDF; the code below predicts the performance score for RAM = 75.)
5. Plot residuals against fitted values.
6. Generate a Normal Q-Q plot.
7. Generate a histogram of residuals.

```
cat('\014')
ram<-c(4,6,8,10,12,14,16,18)
pscore<-c(45,52,60,67,73,78,84,88)
df<-data.frame(ram,pscore)
model<-lm(pscore~ram)
print(coef(model))
```

```
ans<-predict(model,data.frame(ram=75))
cat("\nPredicted Website Loading Time when 75 users access the website
simultaneously = ",ans)

fit<-fitted(model)
res<-residuals(model)

plot(fit,res,
     main="Residuals vs Fitted Values",
     xlab="Fitted Values",
     ylab="Residuals")
abline(h=0)

qqnorm(res)
qqline(res)
```

Q.4. cars dataset

Using the inbuilt cars dataset, a researcher wants to predict the stopping distance of cars based on their speed.

Tasks:

1. Create the regression model.
2. Predict stopping distance for speeds of 10, 15, 20 and 25 mph.
3. Store the predictions in a data frame.
4. Display the predictions.
5. Plot the original observations and regression line.
6. Generate the four standard diagnostic plots using plot(model).

```
cat('\014')
data(cars)
model<-lm(dist~speed,data=cars)
Speed<-c(10,15,20,25)
Stopping_Distance<-predict(model,data.frame(speed=x))
df<-data.frame(Speed,Stopping_Distance)
print(df)
plot(cars$speed,cars$dist,
     main="Stopping Distance vs Speed",
     xlab="Speed (mph)",
     ylab="Stopping Distance (miles)",
     )
abline(model)
plot(model)
```

Q.5. airquality dataset

The airquality dataset contains daily air quality measurements in New York. It includes Ozone concentration and Temperature. An environmental researcher wants to determine whether temperature can be used to predict ozone concentration.

Perform following tasks in R:

1. Load the airquality dataset.
2. Check for missing values.
3. Remove observations containing missing values for Ozone and Temp.
4. Fit a simple linear regression model with Ozone as the dependent variable and Temp as the independent variable.
5. Display the model summary.
6. Predict ozone concentration when the temperature is 80°F.
7. Plot the regression line.

```
cat('\014')
data(airquality)
head(airquality)
df<-na.omit(airquality[, c("Ozone", "Temp")])
model<-lm(Ozone~Temp,data=df)
print(summary(model))
ans<-predict(model,data.frame(Temp=80))
cat("\nOzone concentration when the temperature is 80°F = ",ans)
plot(df$Temp, df$Ozone,
     main = "Ozone Concentration vs Temperature",
     xlab = "Temperature (°F)",
     ylab = "Ozone Concentration",
     pch = 19)
abline(model)
```

Practical No. 05 – Weighted Least Square using R

Q.1. Study hours vs marks (WLS)

Consider the following manually collected data of students and perform Weighted Least Squares regression to study the relationship between Study Hours (X) and Marks (Y).

Study Hours	Marks	Weight
2	45	1
3	50	2
4	58	3
5	65	4
6	72	5

Fit the WLS regression model and predict marks for a student who studies 7 hours.

```
cat('\014')
study_hours<-c(2, 3, 4, 5, 6)
marks<-c(45, 50, 58, 65, 72)
weight<-c(1, 2, 3, 4, 5)

df<-data.frame(study_hours, marks, weight)
```

```
print(df)

#WLS model
model<-lm(marks~study_hours,data=df,weights=weight)

#Model summary
print(summary(model))

#Regression equation
eq<-coef(model)
b0<-eq[1]
b1<-eq[2]
cat("\nRegression Equation: \nMarks =",b0,"+",b1,"*Study Hours\n")

# Prediction
answer<-predict(model,data.frame(study_hours=7))
cat("\nPredicted Marks=",answer)

# Plot
plot(study_hours, marks,
     main = "WLS Regression: Marks vs Study Hours",
     xlab = "Study Hours",
     ylab = "Marks",
     pch = 19)

abline(model)
```

Q.2. Advertising expenditure vs sales (WLS)

A shop records the following data for Advertising Expenditure (X) and Sales (Y).

Advertising Expenditure	Sales	Weight
10	25	1
20	38	2
30	55	3
40	70	4
50	88	5

Perform WLS regression and predict sales when advertising expenditure is 35 units.

```
cat('\014')
advertising<-c(10,20,30,40,50)
sales<-c(25,38,55,70,88)
weight<-c(1,2,3,4,5)

df<-data.frame(advertising,sales,weight)
print(df)

#WLS model
model<-lm(sales~advertising,data=df,weights=weight)

#Model summary
print(summary(model))

#Regression equation
eq<-coef(model)
b0<-eq[1]
b1<-eq[2]
cat("\nRegression Equation: \nSales =",b0,"+",b1,"*Advertising\n")

# Prediction
answer<-predict(model,data.frame(advertising=35))
cat("\nPredicted Sales=",answer)

# Plot
plot(advertising,sales,
     main = "WLS Regression: Sales vs Advertising",
     xlab = "Advertising",
     ylab = "Sales",
     pch = 19)

abline(model)
```

Q.3. cars dataset (WLS)

Using the built-in cars dataset, perform WLS regression between speed and dist. Use 1/speed as the weight. Predict stopping distance for a speed of 20 mph.

```
cat('\014')
data(cars)
print(head(cars))

#Create weights
w<-1/cars$speed

#Fit WLS model
wls_model<-lm(dist~speed,data=cars,weights=w)

#Model summary
print(summary(wls_model))

#Regression equation
eq<-coef(wls_model)
cat("\nRegression Equation: dist =",b0, "+",b1,"*speed\n")

#Prediction for speed = 20
answer<-predict(wls_model,data.frame(speed=20))
cat("\nPredicted Speed = ",answer)

# Plot
plot(cars$speed, cars$dist,
     main = "WLS Regression: Stopping Distance Vs Speed",
     xlab = "Speed",
     ylab = "Stopping Distance",
     pch = 19)

abline(wls_model)
```

Q.4. trees dataset (WLS)

Using the built-in trees dataset, perform WLS regression between Girth and Volume. Use 1/Height as the weight. Predict volume for a tree having a girth of 15 inches.

```
cat('\014')
data(trees)
print(head(trees))

#Create weights
w<-1/trees$Height
```

```
#Fit WLS model
wls_model<-lm(Volume~Girth,data=trees,weights=w)

#Model summary
print(summary(wls_model))

#Regression equation
eq<-coef(wls_model)
cat("\nRegression Equation: Volume =",b0, "+",b1,"*Girth\n")

#Prediction for Girth = 15
answer<-predict(wls_model,data.frame(Girth=15))
cat("\nPredicted Volume = ",answer)

# Plot
plot(trees$Girth, trees$Volume,
     main = "WLS Regression: Volume Vs Girth",
     xlab = "Girth",
     ylab = "Volume",
     pch = 19)

abline(wls_model)
```

Practical No. 06 – Multiple Linear Regression and Ridge Regression using R

Q.1. Exam marks (Multiple Linear Regression)

Create a dataset containing Study Hours, Attendance, Assignment Marks and Exam Marks for 10 students. Build a Multiple Linear Regression model to predict Exam Marks if Study Hours = 8, Attendance = 85, Assignment = 82.

Student	Study Hours	Attendance	Assignment	Exam Marks
1	2	65	55	52
2	3	70	60	58
3	4	72	65	64
4	5	75	70	69

Student	Study Hours	Attendance	Assignment	Exam Marks
5	6	78	72	74
6	7	82	78	81
7	8	85	80	85
8	9	88	85	89
9	10	92	88	94
10	11	95	92	97

```
cat('\014')
student_data<-data.frame(
  StudyHours=c(2,3,4,5,6,7,8,9,10,11),
  Attendance=c(65,70,72,75,78,82,85,88,92,95),
  Assignment=c(55,60,65,70,72,78,80,85,88,92),
  ExamMarks=c(52,58,64,69,74,81,85,89,94,97)
)
print(student_data)

# Multiple Linear Regression
model<-lm(ExamMarks~StudyHours+Attendance+Assignment,data=student_data)

#Display model summary
print(summary(model))

#Prediction
new_student<-data.frame(StudyHours = 8,Attendance = 85,Assignment = 82)
answer<-predict(model,newdata=new_student)
cat("\nPredicted Marks=",answer)
```

Q.2. House price (Multiple Linear Regression)

Create a dataset containing House Area, Number of Bedrooms, Age of House and House Price. Use Multiple Linear Regression to predict the price of a house with Area = 1900, Bedrooms = 4, Age = 6.

Area	Bedrooms	Age	Price
800	2	20	45

Area	Bedrooms	Age	Price
1000	2	18	55
1200	3	15	65
1400	3	12	75
1600	3	10	88
1800	4	8	100
2000	4	7	115
2200	4	5	125
2400	5	4	140
2600	5	2	155

```
cat('\014')
HousePrice<-data.frame(
  Area=c(800,1000,1200,1400,1600,1800,2000,2200,2400,2600),
  Bedrooms=c(2,2,3,3,3,4,4,4,5,5),
  Age=c(20,18,15,12,10,8,7,5,4,2),
  Price=c(45,55,65,75,88,100,115,125,140,155)
)
print(HousePrice)

# Multiple Linear Regression
model<-lm(Price~Area+Bedrooms+Age,data=HousePrice)

#Display model summary
print(summary(model))

#Prediction
new_house<-data.frame(Area=1900,Bedrooms=4,Age=6)
answer<-predict(model,newdata=new_house)
cat("\nPredicted Price=",answer)
```

Q.3. mtcars dataset (Multiple Linear Regression)

Using the inbuilt mtcars dataset, create a Multiple Linear Regression model to predict Miles Per Gallon (mpg) using: Horsepower (hp), Weight (wt), Displacement (disp).

Also predict the MPG of a car having: hp = 120, wt = 2.8, disp = 200.

```
cat('\014')
data(mtcars)
print(head(mtcars))

# Create Multiple Linear Regression model
model<-lm(mpg~hp+wt+disp,data = mtcars)

# Display model summary
print(summary(model))

#Prediction
new_car<-data.frame(hp=120,wt=2.8,disp=200)
answer<-predict(model,newdata=new_car)
cat("\nPredicted MPG=",answer)
```

Q.4. mtcars dataset (Ridge Regression)

Using the mtcars dataset, apply Ridge Regression to predict mpg using:

- hp
- wt
- disp
- qsec

Use cross-validation to determine the optimum value of lambda.

```
# Install package if required
# install.packages("glmnet")

cat('\014')
library(glmnet)
#Load dataset
data(mtcars)
#Create predictor matrix
x<-as.matrix(mtcars[, c("hp", "wt", "disp", "qsec")])

#Dependent variable
y<-mtcars$mpg

#Ridge Regression
```

```
ridge_model<-glmnet(x,y,alpha=0)

#Display model
#print(ridge_model)

#Cross-validation
cv_ridge<-cv.glmnet(x,y,alpha=0)

#Display optimum lambda
cat("\n Optimum Lambda=",cv_ridge$lambda.min)

#Plot cross-validation result
plot(cv_ridge)

#Coefficients at optimum lambda
cat("\nCoefficients\n")
print(coef(cv_ridge,s="lambda.min"))
```

Q.5. airquality dataset (Ridge Regression)

Using the inbuilt airquality dataset, predict Ozone using:

- Solar.R
- Wind
- Temp

Apply Ridge Regression and determine the optimum lambda using cross-validation.

```
# Install package if required
# install.packages("glmnet")

cat('\014')
# Load package
library(glmnet)
# Load inbuilt dataset
data(airquality)
#Remove missing values
air_data<-na.omit(airquality)
#Predictor matrix
x<-as.matrix(air_data[, c("Solar.R", "Wind", "Temp")])
#Dependent variable
y<-air_data$Ozone
#Ridge Regression
```

```
ridge_model<-glmnet(x,y,alpha=0)
#Display model
#print(ridge_model)
#Cross-validation
cv_ridge<-cv.glmnet(x,y,alpha=0)
#Optimum lambda
cat("\nOptimum Lambda = ",cv_ridge$lambda.min)
# Plot cross-validation
plot(cv_ridge)
# Display coefficients
cat("\nCoefficients\n")
print(coef(cv_ridge,s="lambda.min"))
# Prediction for new observation
new_data<-matrix(c(200, 10, 80),nrow = 1)
colnames(new_data)<-c("Solar.R", "Wind", "Temp")
answer<-predict(cv_ridge,newx = new_data,s="lambda.min")
cat("\nPredicted Ozone Concentration = ",answer)
```